Project: Predicting the Activity of a Molecule

Data Understanding

Dataset Overview

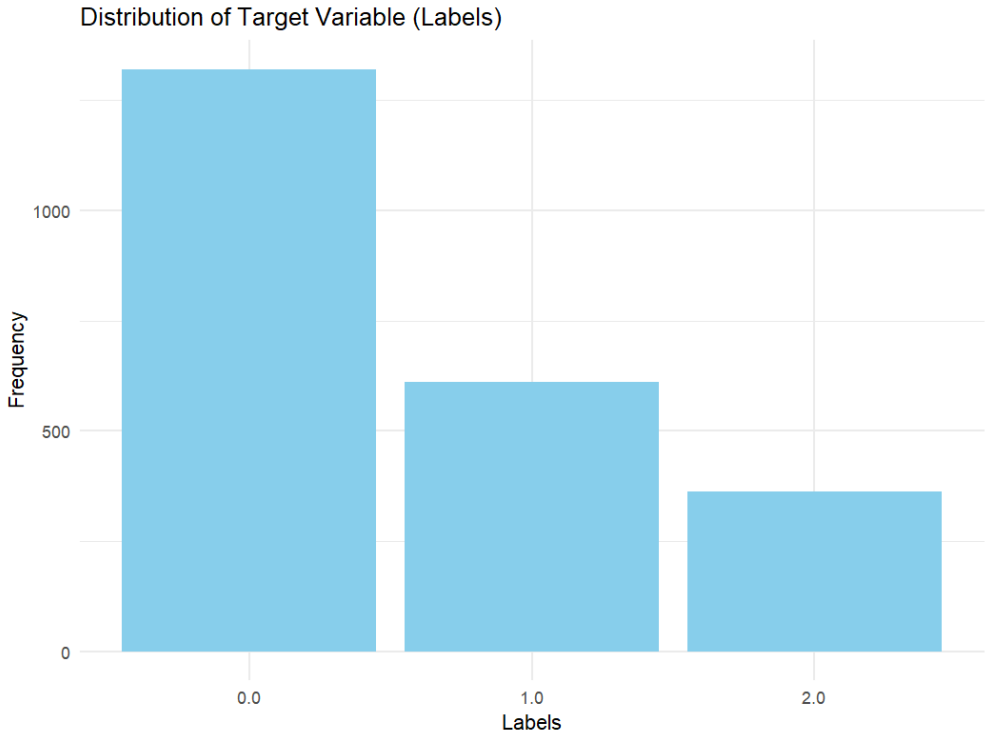
- **2 594** molecules × **169** input variables (features).
- Aside from the index, only the **Labels** column was non-numeric.

Feature Summaries

- **Average values** for most features are very close to zero → most bits aren't "on" very often.
- **Variances** are low for most features, though a subset shows enough spread to be useful.

Target Distribution

Class	Count	Percentage
not active (0)	1,506	58.1%
weakly active (1)	674	26.0%
strongly active (2)	414	15.9%



Data Cleaning

Index and Duplicates

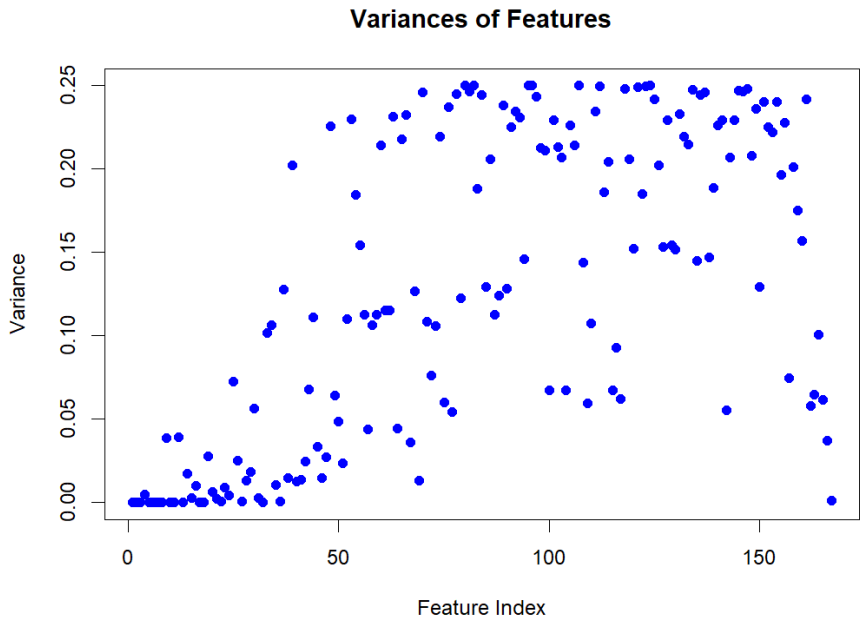
- The initial index column labeled **"Column1" was dropped**.
- A total of **301 exact duplicates** were identified, leaving 2,293 unique molecules.

Missing Values and Zero-Variance Features

- **No missing values** were found across all features.
- **Nine features with zero variance**, such as ISOTOPE was removed .

These steps reduced the number of features from 169 to 159.

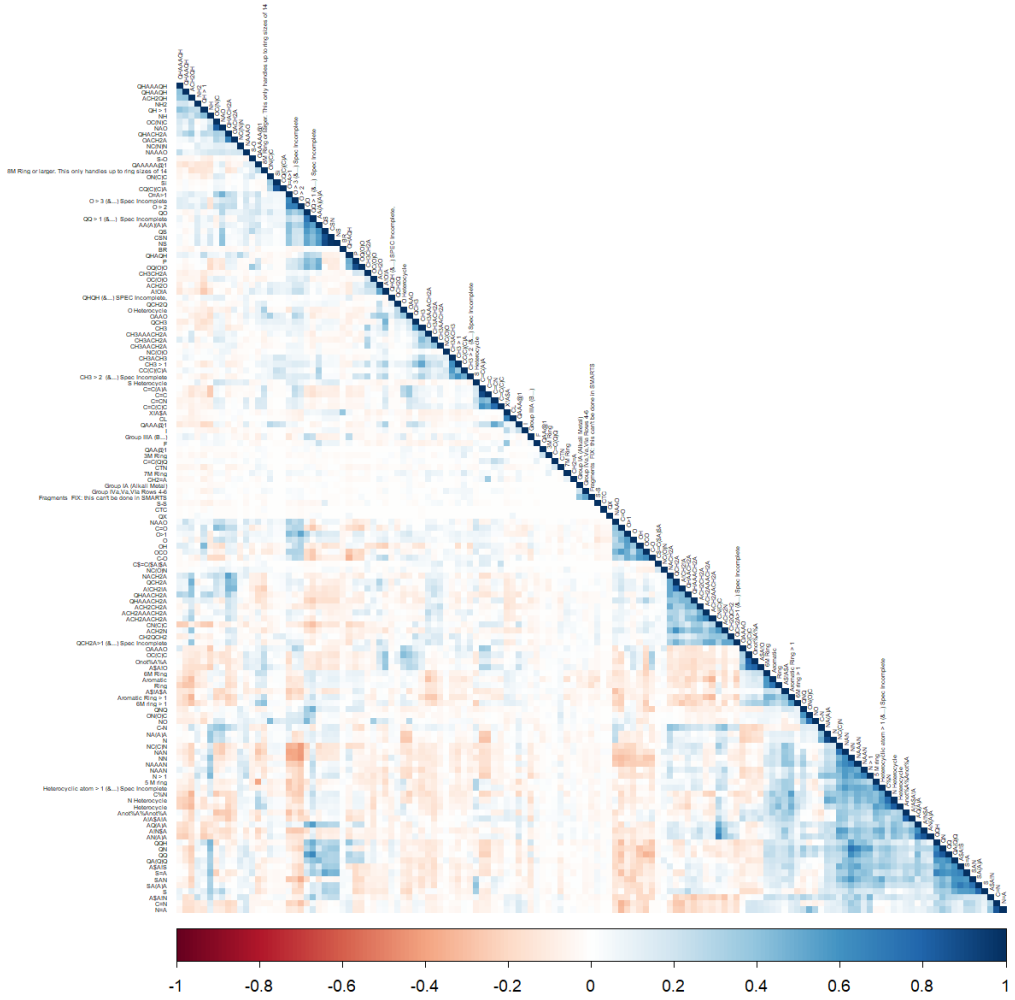
Visualizing the variances with a plot



Correlation Filtering

- The Pearson correlations were computed for the remaining features.
- **Twenty Features with a correlation greater than 0.9 were removed**, except for those that showed strong correlation with the target variable
- The feature "NS" was retained because it demonstrated significant correlation ($|r| > 0.20$) with the target variable.
- The dataset now has 139 features after cleaning.

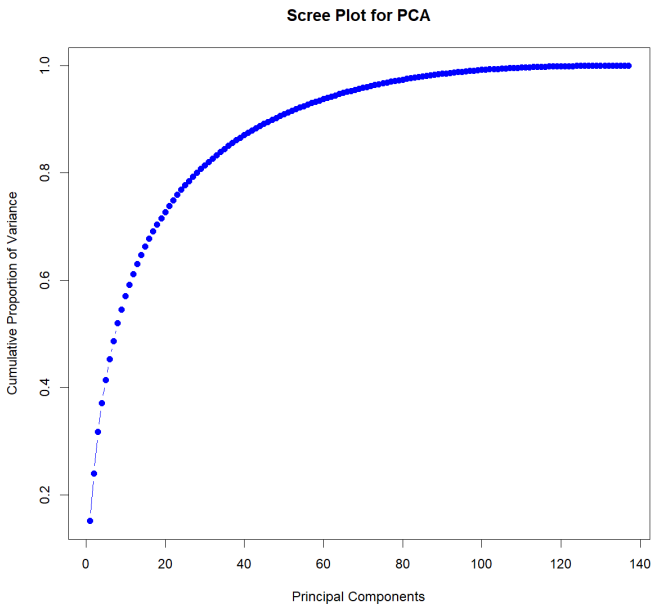
Below there is a correlation plot with the features remaining



Principal Component Analysis (PCA)

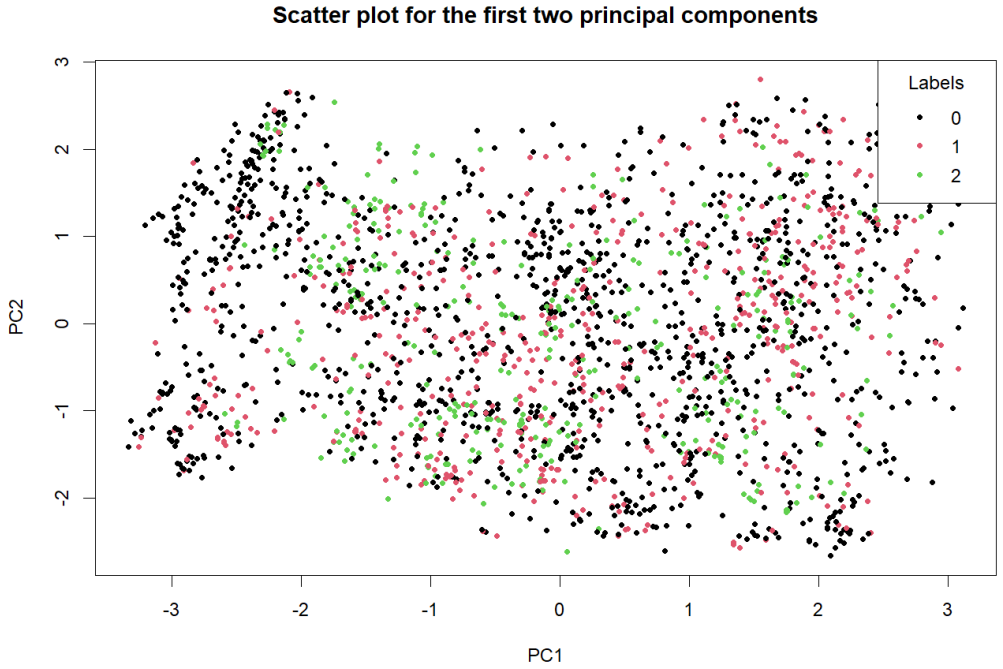
Exploring PCA Components: We conducted PCA on the dataset and examined the explained variance. The first two components failed to provide significant separation among the target classes, suggesting that molecular activity is not easily captured in a low-dimensional space.

- Scree plot showing cumulative variance explained.

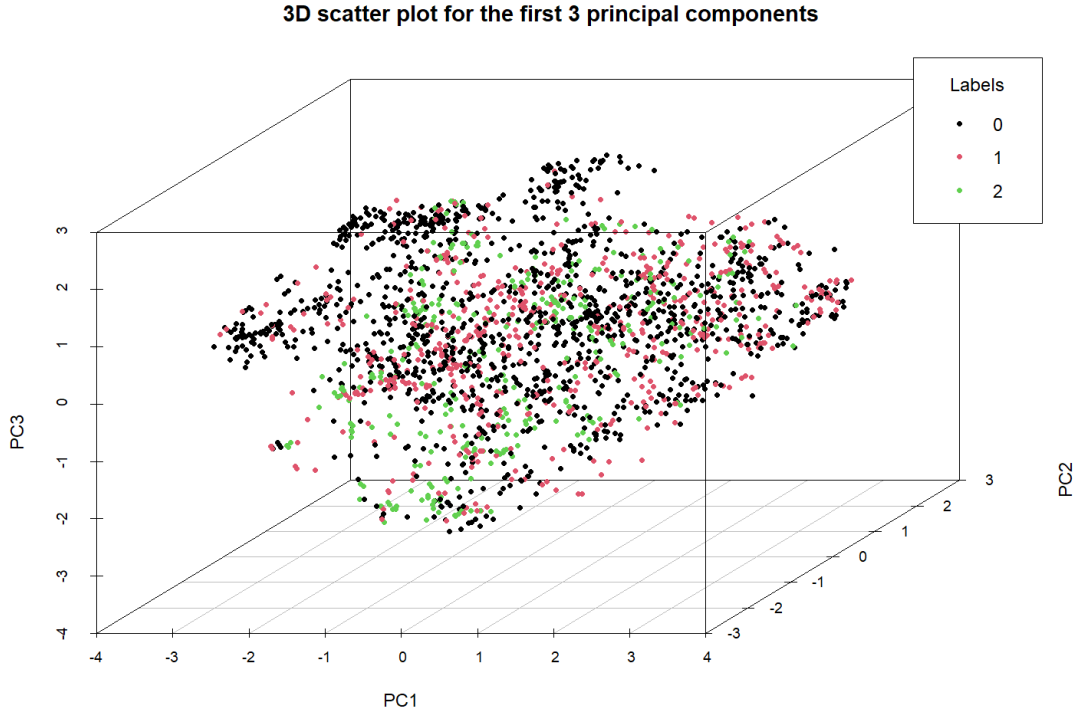


Comment: The scree plot shows that many components are required to explain 95% of the variance, indicating that PCA is not effective for dimensionality reduction in this dataset.

- 2D PCA scatterplot colored by class labels.



- 3D PCA scatterplot to visualize overlap.



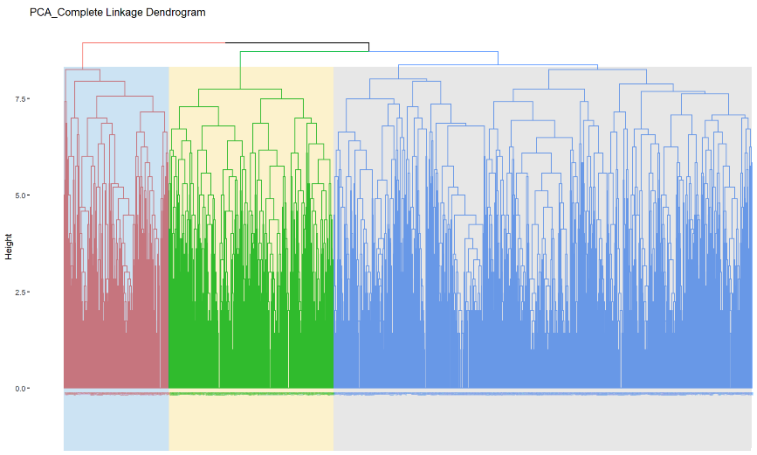
Conclusion: PCA is not effective for this dataset due to overlap between Labels.

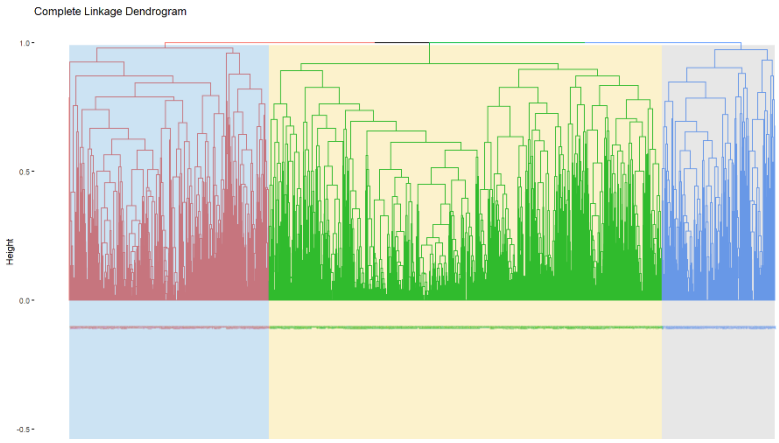
Clustering Analysis

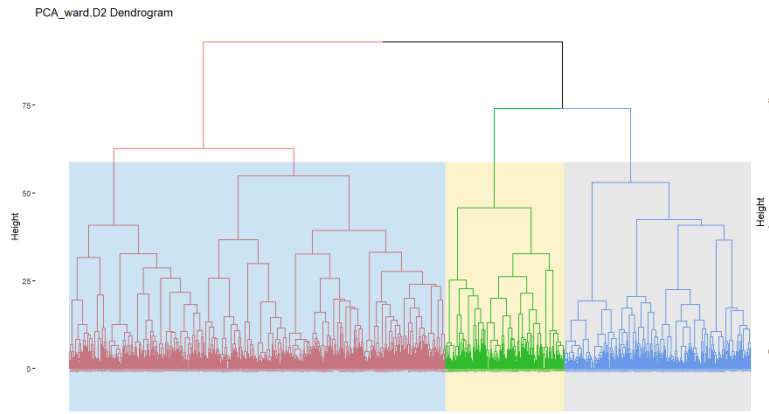
Hierarchical Clustering:

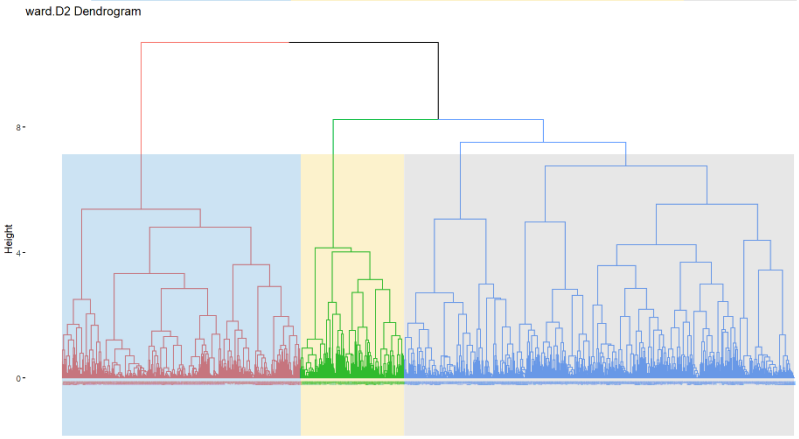
We performed hierarchical clustering using Jaccard distance and Ward's method and one more to time using PCA with Euclidean distance and complete method for comparison. Cluster assignments were compared to the true labels.

- Dendrograms for hierarchical clustering.

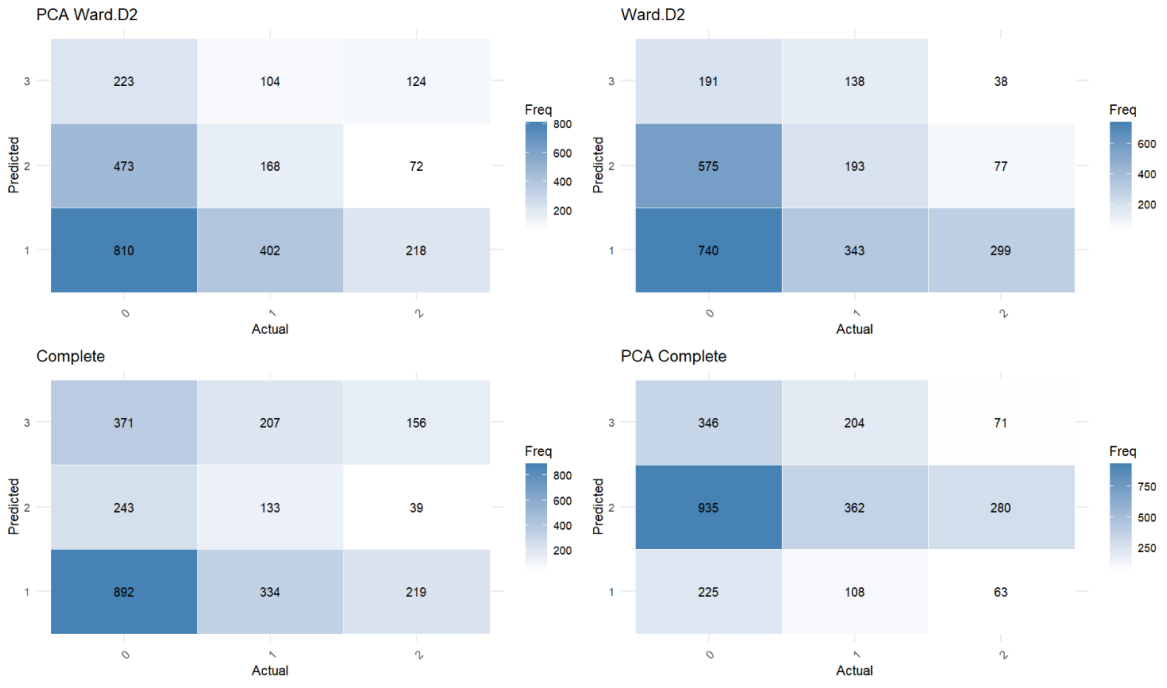




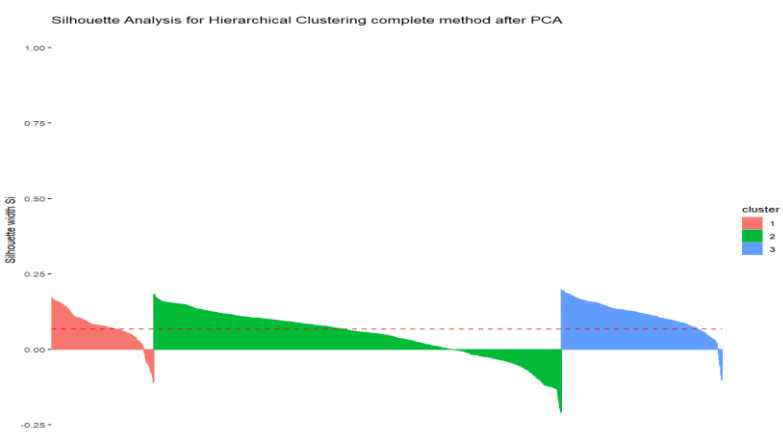


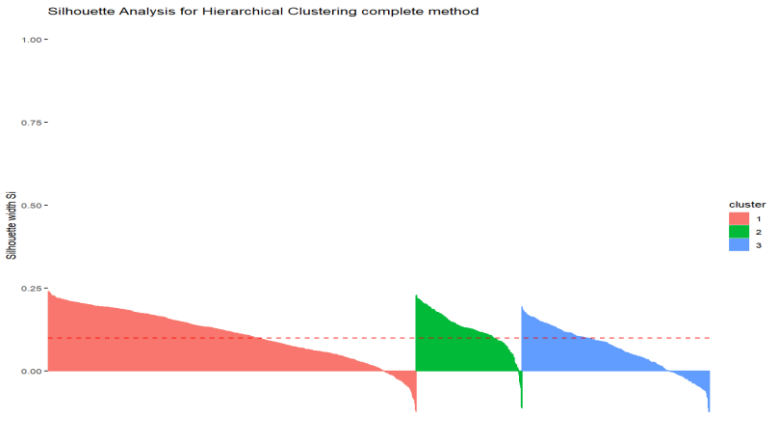


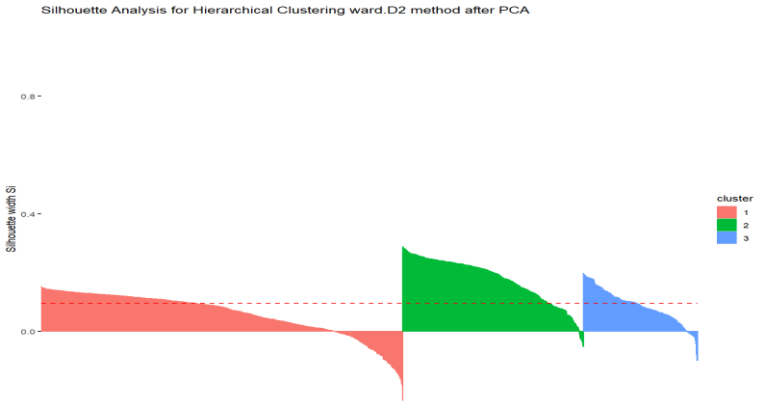
- Confusion matrices for every method with Labels

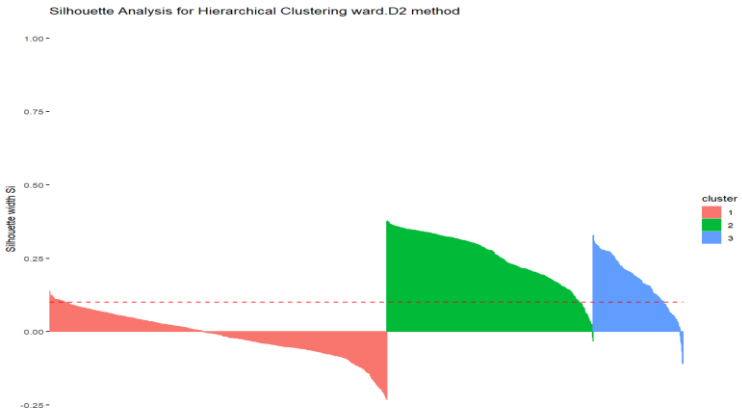


- Silhouette plots









Conclusions about the hierarchical clustering analysis:

1. The confusion matrices show that no cluster clearly matches a single true label (0, 1, 2). In other words, each of the three clusters contains a mixture of actual class labels. So the clusters we get from hierarchical clustering do not map onto the known activity labels in the dataset.

2. Ward.D2 with Jaccard might gather one or two groups that mostly share one label, but also "steals" points from other labels, lowering purity. Ward.D2 with PCA tries to cluster on the major principal component directions, sometimes leading to slightly different label distributions in each cluster, but also exhibiting overlap. Complete Linkage has a similar pattern: each cluster is a mix of 0, 1, 2.

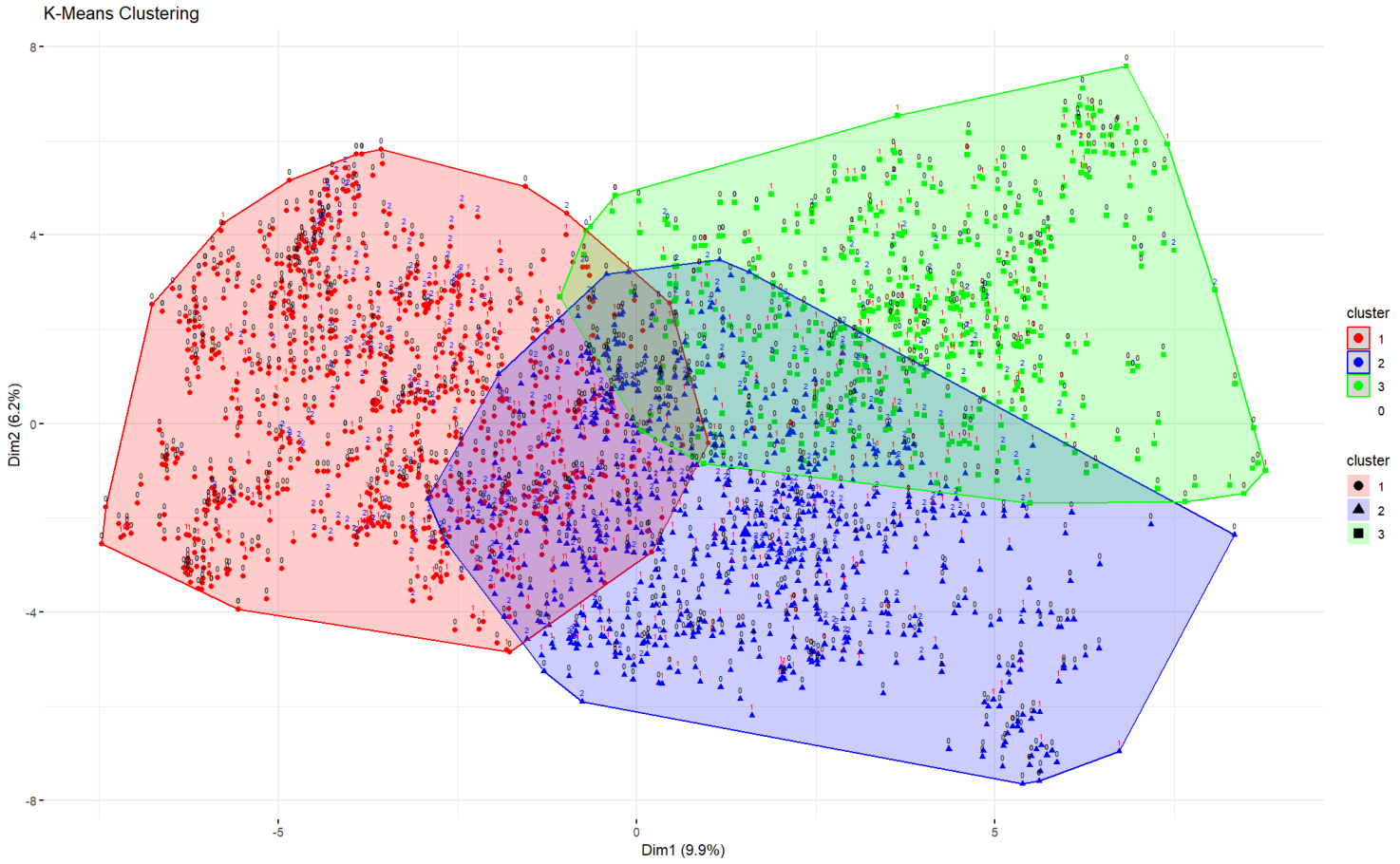
3. All methods yield silhouette widths mostly in the 0.0–0.25 range, suggesting weak separation. Values that approach or exceed 0.5 typically indicate well-defined clusters. Here, the dataset does not separate neatly into three distinct clusters with either the original features (using Jaccard) or the PCA-transformed features (using Euclidean)

4. For Ward.D2 on Jaccard, one cluster did achieve ~0.25, which is the best of the scores. This indicates that for binary fingerprint data, Jaccard is often more appropriate than Euclidean on PCA scores.

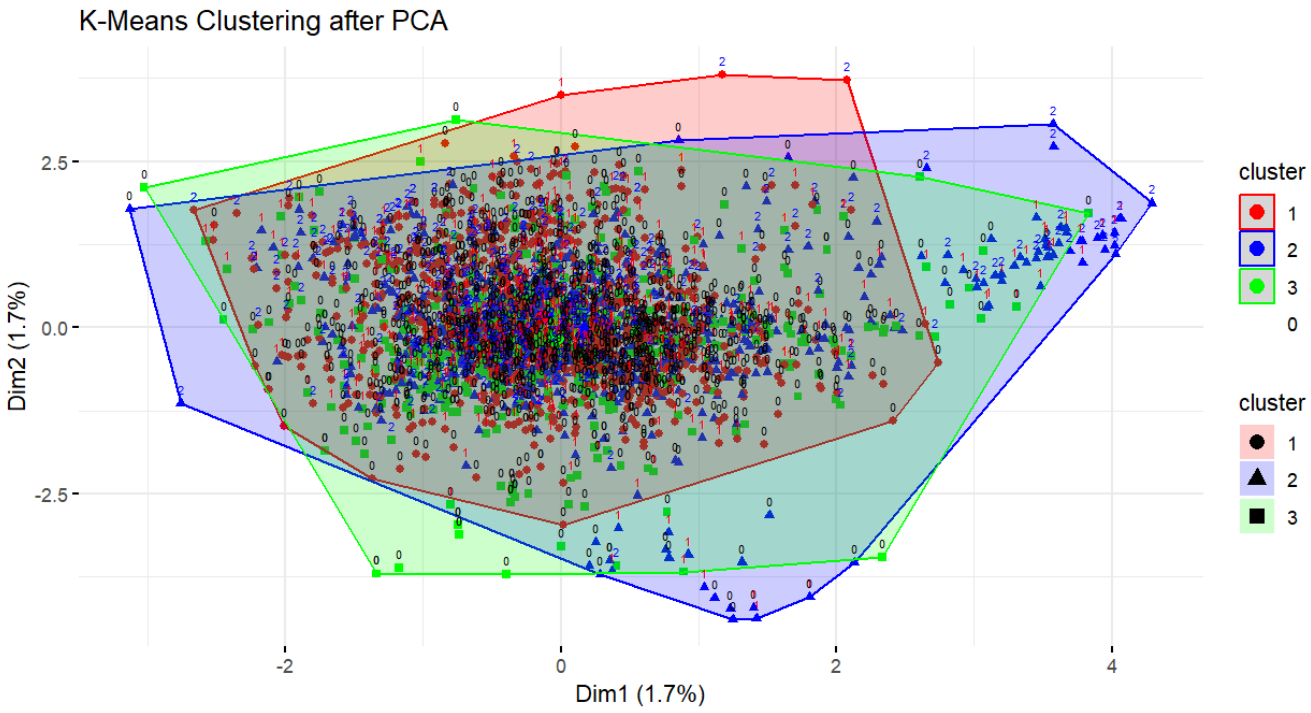
5. Whether we use Ward.D2 or Complete Linkage, with PCA (Euclidean) or without PCA (Jaccard), the hierarchical clustering solution has weak consistency with the known labels and low average silhouette values.")

K-Means Clustering:

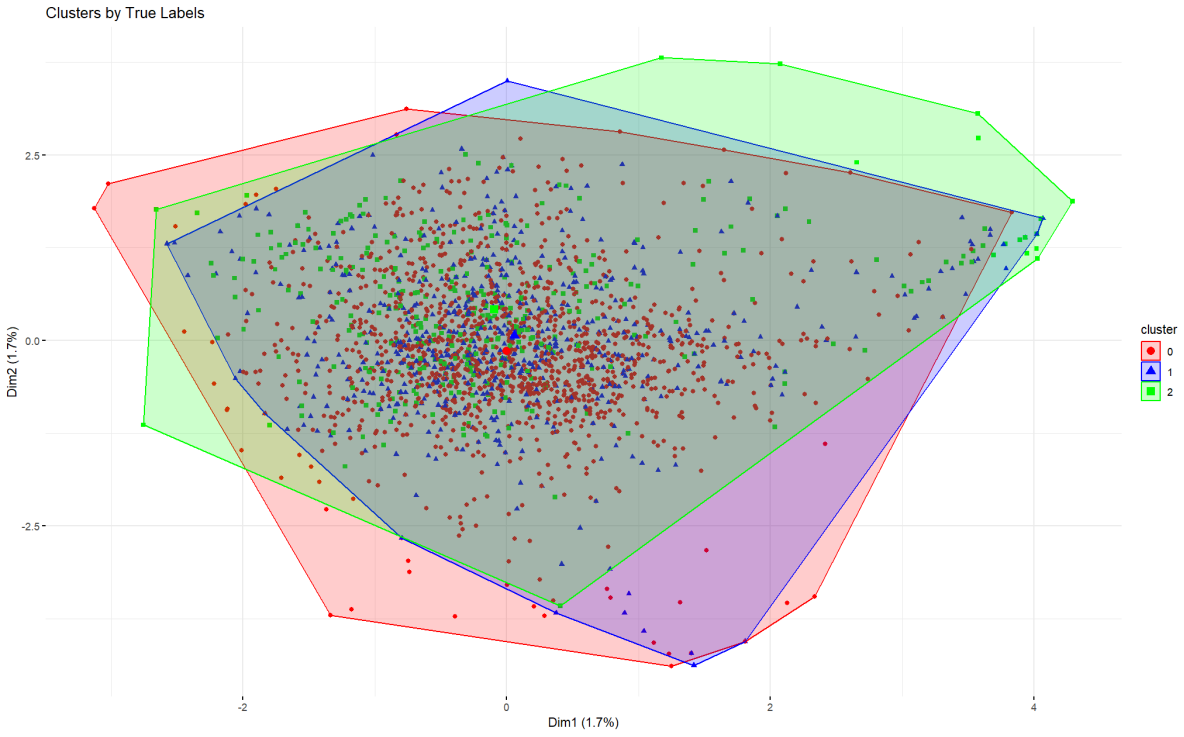
- K-Means clustering (k=3) was applied on original data.



- K-Means was also applied after performing PCA, retaining the first 60 components which explained nearly 95% of the dataset's variance.



- We overlay the true labels on the PCA plot to assess alignment with actual molecular activity.



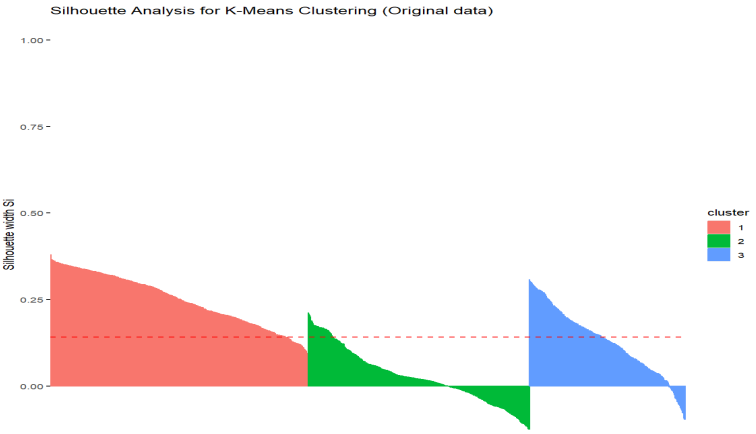
comments:

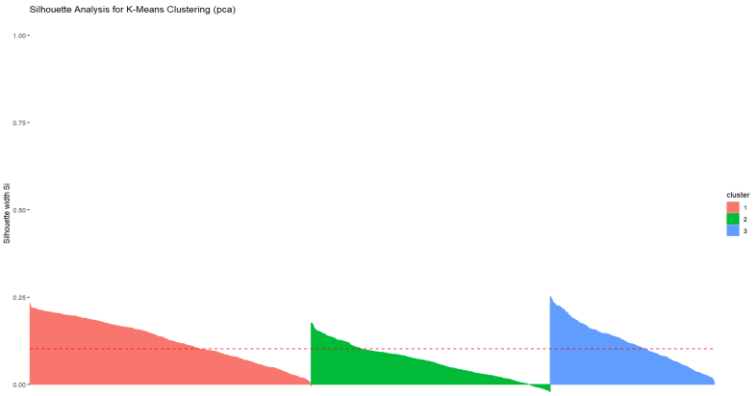
K-Means before PCA performs better than K-Means after PCA. The clusters in the first approach show some degree of separation, but the boundaries are not well-defined. After PCA, the situation worsens, with clusters overlapping even more significantly, almost appearing identical.

In both clustering methods applied, the generated clusters do not perfectly align with the true labels of 0, 1, 2. This mismatch suggests that K-Means alone may not be the most effective clustering approach for this particular dataset. The actual class labels are distributed across

different clusters, indicating that there is only partial separation at best. While some degree of separation is observed, it is not sufficient to clearly distinguish the groups based on the true labels.

- Silhouette plots for K-means follow to test the previous observations





comments:

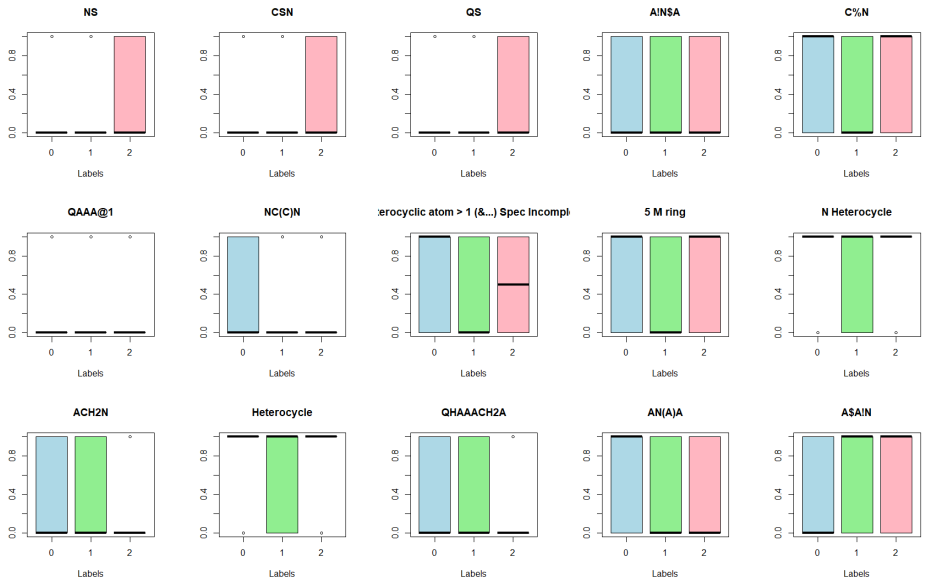
The silhouette results for K-Means clustering, both on the original data and after PCA, do not strongly isolate the three known label groups. Most points have silhouette scores below 0.20, and an average silhouette width of approximately 0.18 indicating overlapping clusters and suggesting that the data may not exhibit a natural 3-cluster structure aligned with the target labels

Clustering without PCA (using Jaccard distance) has a slightly better overall silhouette (~0.14) than clustering with PCA (~0.10). So for binary molecular fingerprints, the Jaccard distance can sometimes capture the data's structure better than Euclidean distance in the PCA-transformed space.

Classification

Feature Significance Testing

We conducted one-way ANOVA for each feature to test for significant differences. We identified the features with the lower p-values so we can use them in the future when needed. A boxplot with the features having the top 15 most significant differences follows.



Comments:

Since the dataset contains three groups (0, 1, 2), a t-test would not be appropriate for confirming group differences. One-way ANOVA was performed for each feature to check whether there are statistically significant differences between labels.

Features with small p-values (< 0.05) indicate significant variation among groups, meaning they can help classify molecular activity. Top discriminative features found using ANOVA include NS, CSN, QS, A!N$A, C%N, which had extremely small p-values (e.g., p < 1e-30). These features exhibit clear separation between at least one pair of labels, making them crucial for classification.

However, examining the boxplots some features showed overlapping distributions between classes, making them less useful for classification.")

Multinomial Logistic Regression

Model Using All Features:

A multinomial logistic regression model trained on all features achieved **64.86% accuracy and Kappa 0,35.**

The confusion matrix showed that **Class 0 had the highest sensitivity** (82%) **and precision** (72.2%), while **Class 1 was the hardest to classify** (28%).

Model Using Top 15 ANOVA Features:

When trained on only the top 15 features, **accuracy dropped to 56.24% and Kappa 0.09**.

This suggests that reducing features based purely on univariate significance may discard important interactions.

Comments:

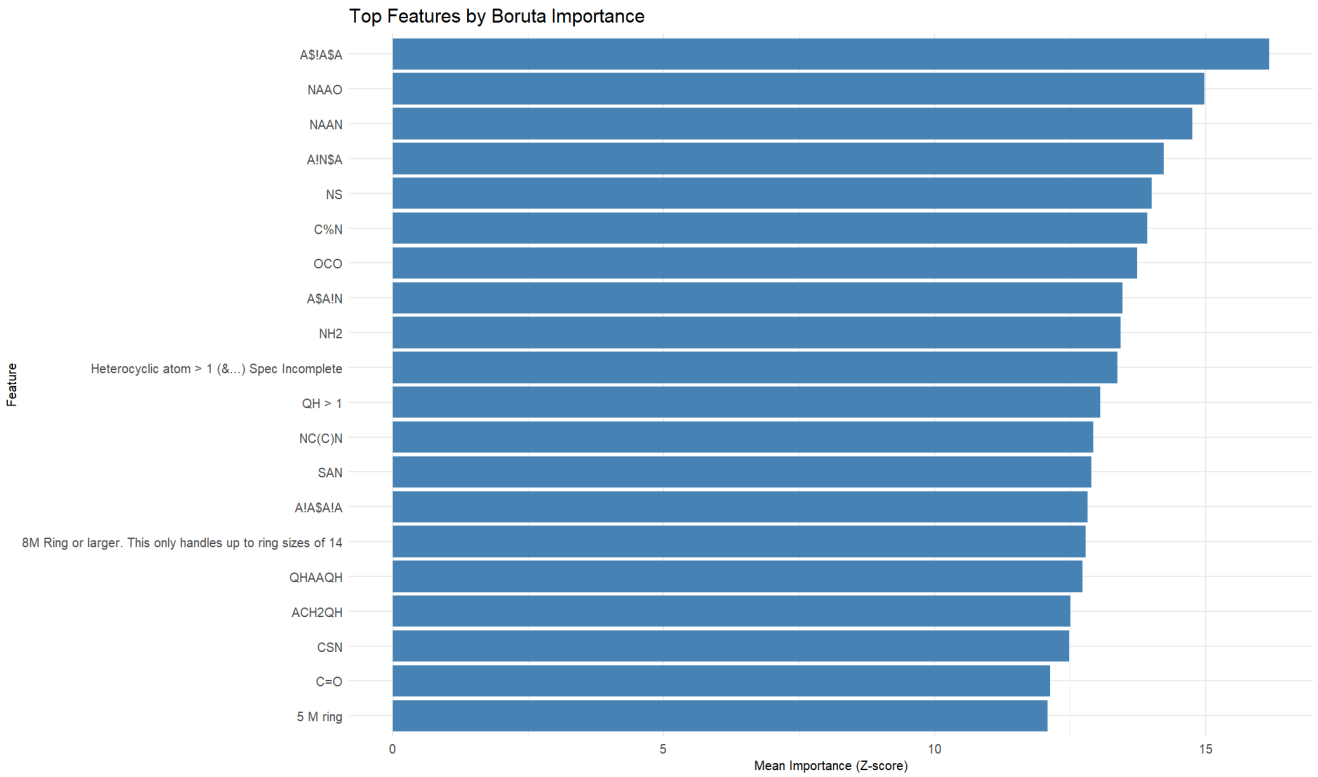
The full model demonstrates superior performance compared to the top 15 ANOVA-selected features model, showing significant differences in accuracy (64.9% vs. 56.2%) and class-specific metrics. To further improve our results, we should address class imbalance (Class 0: 58%, Class 1: 26%, Class 2: 16%) and consider employing advanced feature selection methods such as LASSO, RFE, or Boruta. We can then re-perform Multinomial Logistic Regression to evaluate the outcomes.

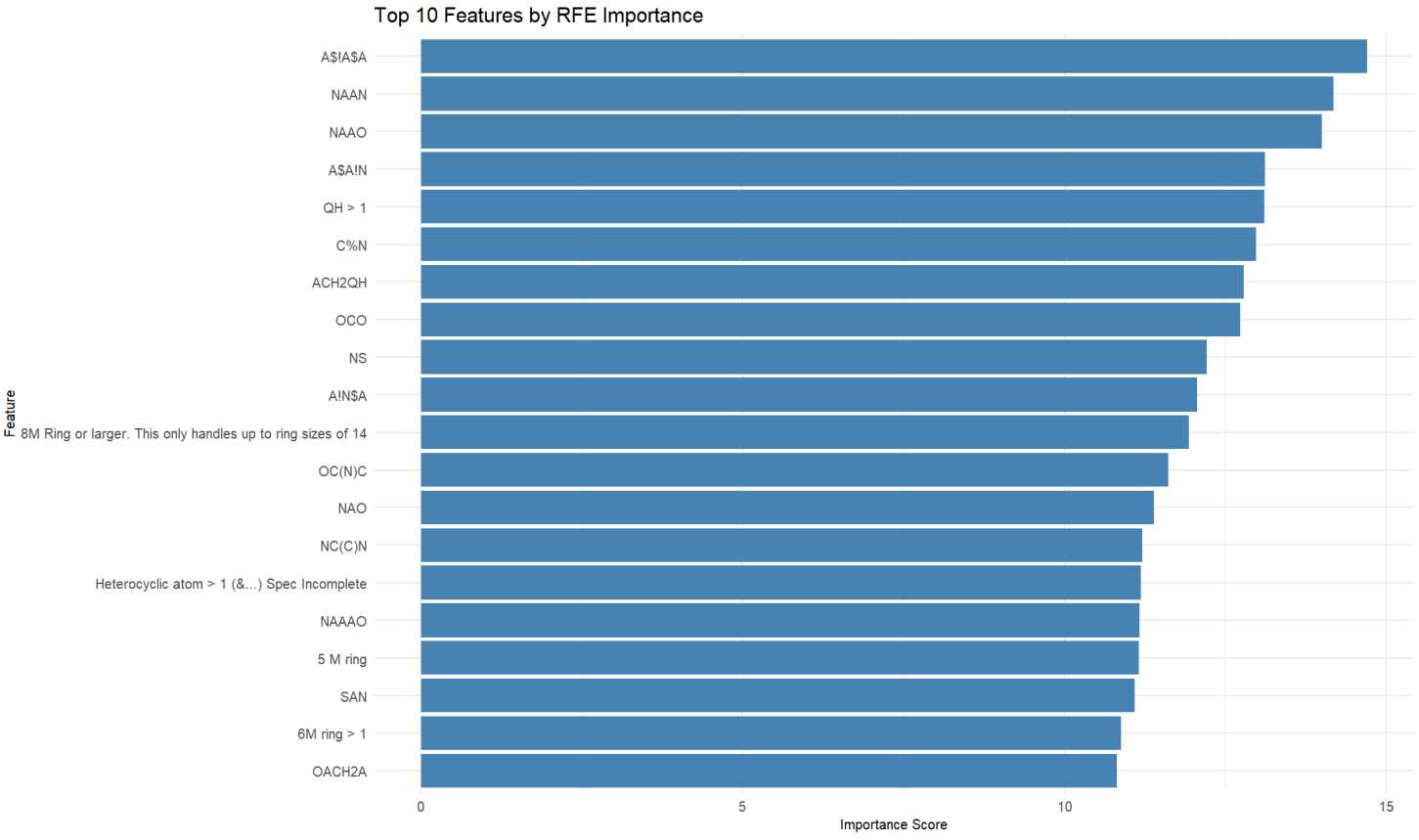
Reducing the feature set to only 15 variables may result in the loss of important information, which could lead to decreased classification performance. Additionally, it has been observed that **reducing the number of features can create duplicate rows in the dataset**, resulting in reduced diversity of entries and information loss.

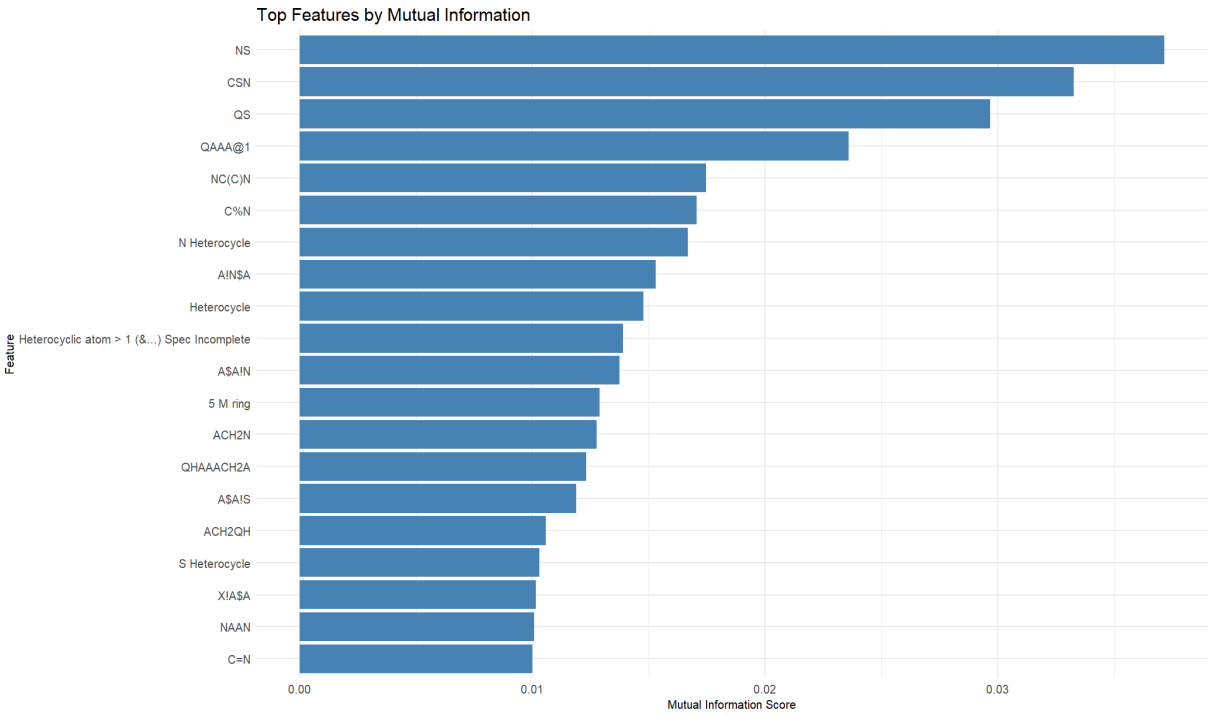
Therefore, to better identify the optimal feature subset, we proceed with applying **four different feature selection methods (Lasso, Boruta, RFE, and Mutual Information)**.

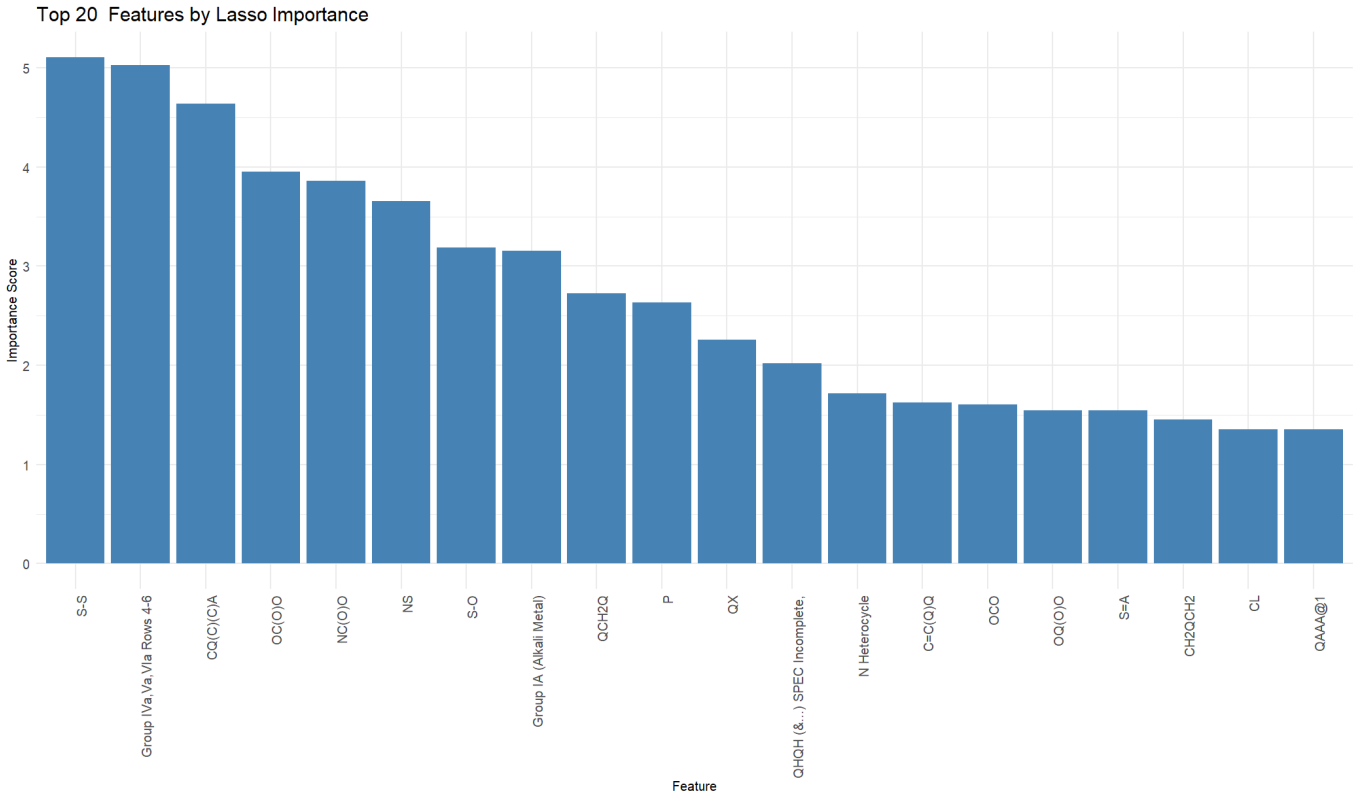
By evaluating models built with different numbers of top mutual features, we aim to determine the optimal balance between dimensionality reduction and model effectiveness.

Performing feature selection with LASSO, Boruta, RFE and Mutual Information we kept just for presentation the top 20 features for each method.


Top Features by Boruta Importance
A$!A$A
NAAO
NAAN
A!N$A
NS
C%N
OCO
A$A!N
NH2
Heterocyclic atom > 1 (&...) Spec Incomplete
QH > 1
NC(C)N
SAN
A!A$A!A
8M Ring or larger. This only handles up to ring sizes of 14
QHAAQH
ACH2QH
CSN
C=O
5 M ring
Feature
0
5
10
15
Mean Importance (Z-score)


Top 10 Features by RFE Importance
A$!A$A
NAAN
NAAO
A$A!N
QH > 1
C%N
ACH2QH
OCO
NS
A!N$A
8M Ring or larger. This only handles up to ring sizes of 14
OC(N)C
NAO
NC(C)N
Heterocyclic atom > 1 (&...) Spec Incomplete
NAAAO
5 M ring
SAN
6M ring > 1
OACH2A
Feature
0
5
10
15
Importance Score





We took the top 120 features from each method (Lasso, Boruta, RFE, and Mutual Information) and put them all together in one list.

Then we counted how many times each feature appeared across the methods. The features that showed up more often are likely more important.

We will use these mutual features in the next steps to test different feature counts and find out how many features give us the best results.

We trained a multinomial logistic regression model using the top 120 mutual features from all selection methods, and we also applied class weights to handle label imbalance. (not active ~ 0.7, weakly active ~ 1.1, strongly active ~ 1.8)

Compared to our earlier models (one with all features and one with only 15 ANOVA-selected features), this one gives better results around 65% accuracy and a Kappa of 0.35.

Model	Accuracy	Kappa	Class 0 Sensitivity	Class 1 Sensitivity	Class 2 Sensitivity	Balanced Accuracy
Full Features	0.6434	0.346	0.8232	0.3005	0.5648	0.671
Top 120 Features, Weighted	**0.6492**	**0.352**	0.8434	0.3005	0.5278	0.6696
Top 15 ANOVA Features	0.6084	0.207	0.899	0.1913	0.25	0.5882

So, selecting 120 important features and adjusting **weights for imbalance really helped improve performance**.

Support Vector Machines (SVM)

Next we trained an SVM model in four different ways to compare results:

1. Using all features with class weights,
2. Using all features without weights,
3. Using top 120 selected features with weights,
4. Using top 120 selected features without weights.

Model SVM	Accuracy	Kappa	Class 0 Sensitivity	Class 1 Sensitivity	Class 2 Sensitivity	Balanced Accuracy
Full Features, Weighted	0.7001	**0.4588**	0.8662	0.3661	0.6574	**0.7257**
Full Features, Unweighted	0.7031	0.4246	0.9419	0.3333	0.4537	0.6892
Top Features, Weighted	0.6943	**0.4539**	0.851	0.4044	0.6111	**0.7225**
Top Features, Unweighted	0.7016	0.4248	0.9394	0.3388	0.4444	0.6891

What we found:

- All models had similar overall accuracy (around 70%).

- The models without weights did a bit better overall in accuracy.

- The models with weights were better at detecting the smaller classes like class 1 and 2, but they made a few more mistakes on class 0.

- Using just the top features worked almost as well as using all features, which is great! it means we can simplify the model without losing much performance.

- Since adding class weights seems to help the models perform better, we will keep using them in future models.

Determining optimal dimensionality reduction.

Our next step involved optimizing the dimensionality reduction process to enhance model performance further. Through various trials, we aimed to identify the most suitable number of features that best balance accuracy, kappa, balanced accuracy, and macro F1-score.

So next, we created two approaches to test how many features give the best results in SVM:

1. Fixed Mutual Ranking:

We merged the top 120 features from all methods, counted how often each feature appeared, and kept the top-n(e.g., top 20, 30, 40, … ,100) from this single merged list for every model.

2. Dynamic Mutual Ranking:

For each n (20, 30, 40, … ,100), we got the top n features from each method and rechecked which features appear in multiple methods. Then we trained a model with those dynamic mutual top n features.

For each approach, we trained SVM models with class weighting ,to handle imbalance, and recorded Accuracy, Kappa, Balanced Accuracy, and Macro F1-score to compare them.

When we use fixed mutual features from the merged top 120 list, the model improves steadily up to 90–100 features, but performance flattens after that.

The dynamic approach, reevaluating the top n most mutual features for each step, consistently performs better.

Best Macro F1 and Accuracy were achieved with the dynamic approach at 70-80 features:

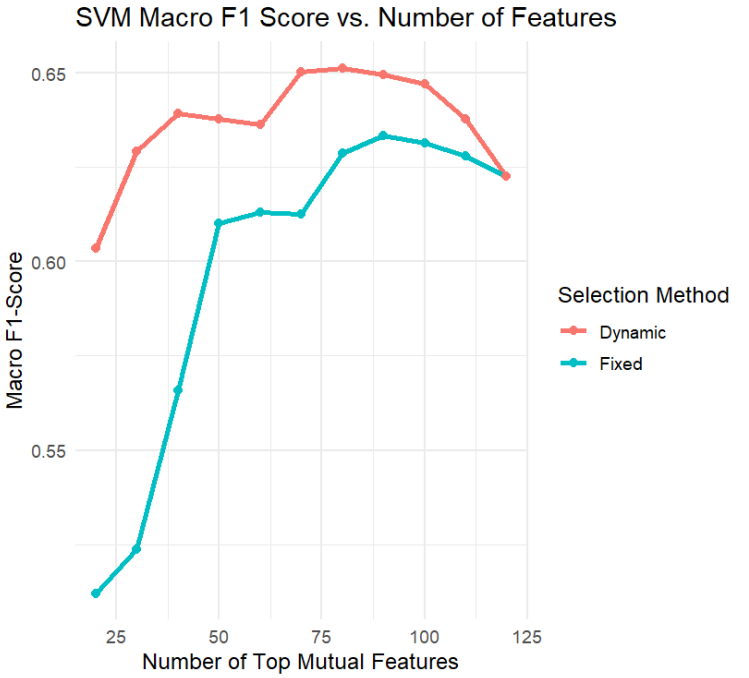
Accuracy:**0.7089**, Macro F1:**0.6503**, Kappa :**0.4898**, Balanced_Accuracy:**0.7475**

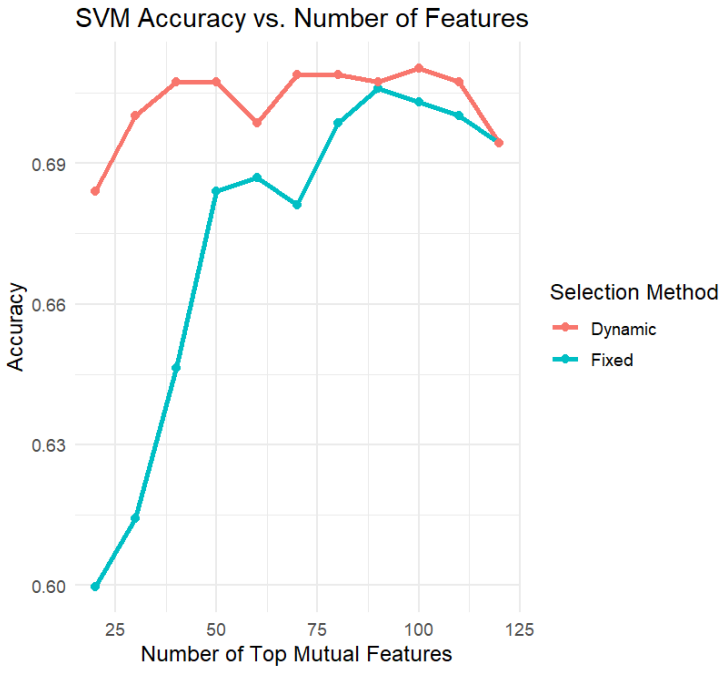
Fixed method had its best at 90-100 features but still underperformed slightly.

So, choosing the best mutual features for each n-size subset gives better performance overall.

So we will keep this strategy to check more models

Plots of comparing F1-score and Accuracy for each number of features we kept





Fixed Mutual

Features	Accuracy	Kappa	Balanced Accuracy	F1 score
20	0.5997	0.2905	0.6458	0.512
30	0.6143	0.3126	0.6547	0.5238
40	0.6463	0.3709	0.6826	0.5657
50	0.6841	0.4336	0.7123	0.6102
60	0.687	0.4388	0.7144	0.613
70	0.6812	0.4343	0.7157	0.6126
80	0.6987	0.4616	0.727	0.6287
90	0.706	0.47	0.7295	0.6334
100	0.7031	0.4684	0.7299	0.6315
110	0.7001	0.4627	0.7264	0.6281
120	0.6943	0.4539	0.7225	0.6226

Dynamic Mutual

Features	Accuracy	Kappa	Balanced Accuracy	F1 score
20	0.6841	0.4242	0.7055	0.6036
30	0.7001	0.4645	0.7321	0.6293
40	0.7074	0.4744	0.7351	0.6392
50	0.7074	0.4791	0.7365	0.6377
60	0.6987	0.4652	0.732	0.6363
70	0.7089	0.4898	0.7475	0.6503
80	0.7089	0.4839	0.744	0.6513
90	0.7074	0.484	0.7434	0.6496
100	0.7103	0.4845	0.7395	0.6472
110	0.7074	0.477	0.7333	0.6377
120	0.6943	0.4539	0.7225	0.6226

Models comparison

Next we tested three classification models : Support Vector Machines (SVM), Multinomial Logistic Regression (MLR), and Random Forest with ranger (RF ranger), using dynamically selected mutual features, trying different numbers of features (from 20 to 120).

Both SVM and Random Forest gave strong results, especially as we increased the number of features.

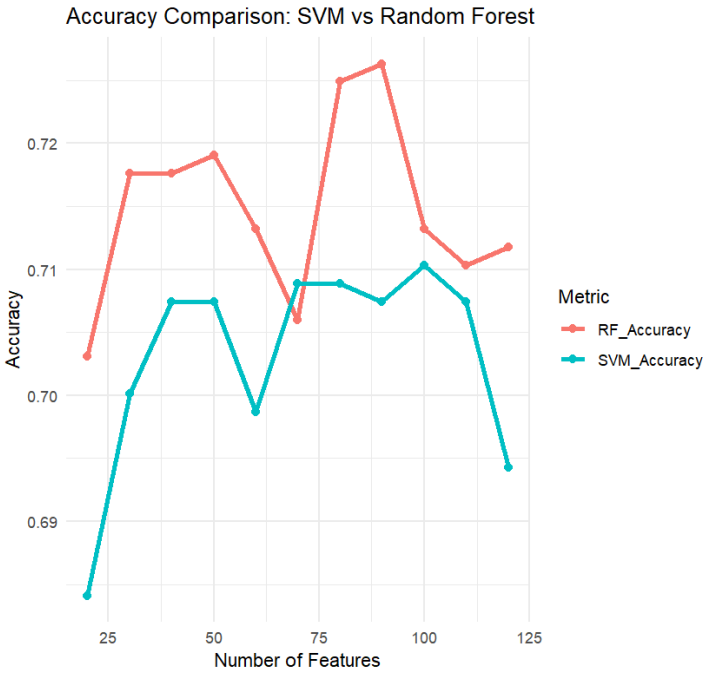
Random Forest, though, generally performed a bit better than SVM, especially in terms of both Accuracy and Macro F1 score.

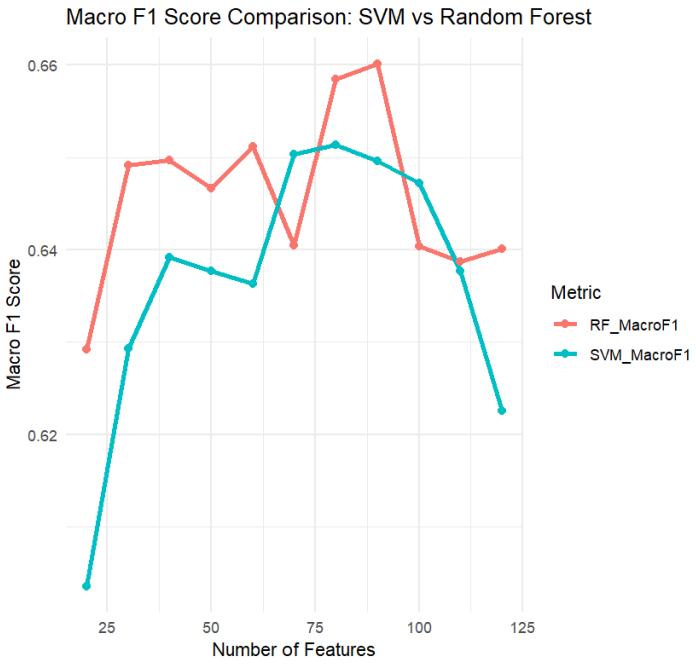
The best performance overall came from Random Forest with 90 features, reaching **72.6%** accuracy, **0.66** Macro F1, **0.5047** Kappa, **0.7457** Balanced Accuracy which makes it the top choice for this task so far.

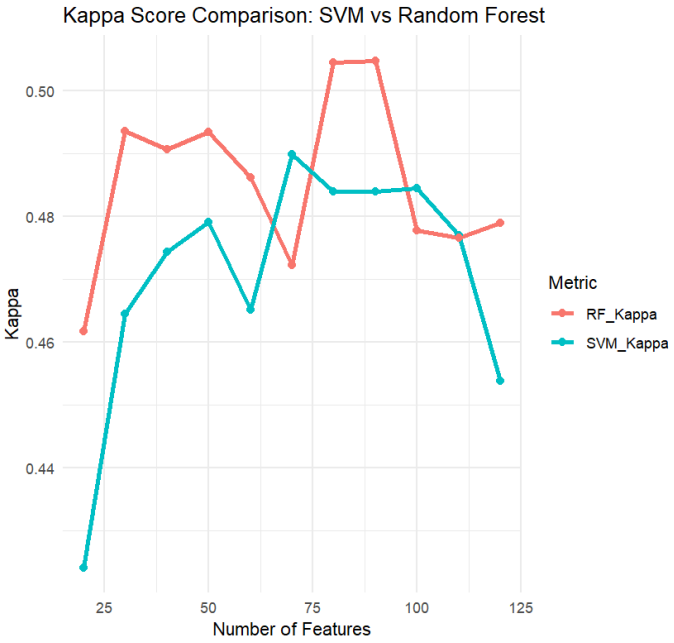
MLR performed worse than the other models. Since MLR does not support class weights here, it was at a disadvantage and will be excluded from the comparison.

Some plots of the models results follows

Classifier	Top Features	Accuracy	Kappa	Balanced Accuracy	Macro F1	Class 0 Sens	Class 1 Sens	Class 2 Sens
SVM	80	0.7089	0.4839	0.7440	0.6513	0.8434	0.4153	0.7130
MLR	90	0.6623	0.3691	0.6762	0.5700	0.8662	0.2951	0.5370
RF (ranger)	90	0.7263	0.5047	0.7457	0.6601	0.8838	0.4262	0.6574







The plots make it super clear that the **random forest algorithm (red line)** totally beats the **Support Vector Machines (SVM) (blue line)** in all metrics.

Association Rules

Each molecule was treated as a transaction, and association rule mining was applied to identify interpretable patterns linking molecular features to activity.

Top rules leading to strong activity (Class 2):

The strongest rules frequently included the NH2 group, often paired with NS, CSN, SAN, or S.Heterocycle, indicating a strong correlation with high activity:

Rule (LHS ⇒ RHS)	Support	Confidence	Lift	Count
{CSN=YES, SAN=YES, NH2=YES} ⇒ Activity=2	0.0135	0.8857	5.5949	31
{NS=YES, SAN=YES, NH2=YES} ⇒ Activity=2	0.0135	0.8857	5.5949	31
{F=YES, A.A.S=YES, NAAO=YES} ⇒ Activity=2	0.0109	0.8621	5.4455	25
{NS=YES, NH2=YES, NAAO=YES} ⇒ Activity=2	0.0188	0.8600	5.4325	43
{CSN=YES, NH2=YES, NAAO=YES} ⇒ Activity=2	0.0153	0.8537	5.3924	35
{CSN=YES, S.Heterocycle=YES, NH2=YES} ⇒ Activity=2	0.0126	0.8529	5.3879	29
{NS=YES, S.Heterocycle=YES, NH2=YES} ⇒ Activity=2	0.0126	0.8529	5.3879	29
{S.Heterocycle=YES, QS=YES, NH2=YES} ⇒ Activity=2	0.0126	0.8529	5.3879	29
{NS=YES, F=YES, OH=YES} ⇒ Activity=2	0.0100	0.8519	5.3810	23

Comments :

These rules exhibit lift values above 5, highlighting meaningful relationships beyond random chance.

NH2 appears consistently in high-confidence rules for activity class 2, underscoring its importance as a feature.

Top rules for inactivity (Class 0):

Some rules predicted inactivity with 100% confidence, though with low support:

- {NC.O.N=YES, QS=NO, AA.A..A.A=NO} ⇒ Activity=0

These patterns provide insight into reliably inactive molecules.

Association Rules with 100% Confidence (Class 0-not active)

Rule (LHS ⇒ RHS)	Support	Confidence	Lift	Count
{NC.N.N=YES, O...2=NO} ⇒ Activity=0	0.0118	1.000	1.737	27
{OC.N.C=YES, C.N.2=NO} ⇒ Activity=0	0.0118	1.000	1.737	27
{NC.O.N=YES,QS=NO,AA.A..A.A=NO}⇒ Activity=0	0.0100	1.000	1.737	23
{NS=NO, NC.O.N=YES, AA.A..A.A=NO} ⇒ Activity=0	0.0100	1.000	1.737	23
{CSN=NO, NC.O.N=YES, AA.A..A.A=NO} ⇒ Activity=0	0.0100	1.000	1.737	23
{NS=NO, NC.O.N=YES, QHAAQH=NO} ⇒ Activity=0	0.0100	1.000	1.737	23
{CSN=NO, NC.O.N=YES, QHAAQH=NO} ⇒ Activity=0	0.0100	1.000	1.737	23
{NC.O.N=YES, QS=NO, CH3ACH3=NO} ⇒ Activity=0	0.0100	1.000	1.737	23
{NC.O.N=YES, CC.C..C.A=NO, QS=NO} ⇒ Activity=0	0.0100	1.000	1.737	23
{NS=NO, NC.O.N=YES, C.N.1=NO} ⇒ Activity=0	0.0105	1.000	1.737	24

References

https://www.geeksforgeeks.org/svm-feature-selection-in-r-with-example/?utm_source

https://machinelearningmastery.com/feature-selection-with-the-caret-r-package/?utm_source

https://www.machinelearningplus.com/machine-learning/feature-selection/?utm_source

https://www.jstatsoft.org/article/view/v077i01

https://www.jstatsoft.org/article/view/v033i01

https://www.jstatsoft.org/article/view/v036i11

chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/http://r.meteo.uni.wroc.pl/web/packages/e1071/e1071.pdf

chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://cran.r-project.org/web/packages/ranger/ranger.pdf?utm_source

https://www.geeksforgeeks.org/svm-feature-selection-in-r-with-example/?utm_source

https://topepo.github.io/caret/recursive-feature-elimination.html?utm_source